

Decentralized Cybersecurity

25 Years of Free Software, dyne:bolic, Netsukuku, and Autonomous Networks

Cybersecurity Boot Camp — Midterm Project

Student: [Your Name]

Instructor: [Instructor Name]

Course: [Course Name]

Date: August 2026

Project Type: Research, Architecture Analysis, and Authorized Laboratory Study

Working premise: cybersecurity can be approached not only by adding defensive controls to centralized systems, but also by examining whether decentralization, free software, user sovereignty, and resilient peer-to-peer architecture can reduce certain classes of dependency and failure.

Abstract

This midterm project examines cybersecurity through the lens of decentralized computing and networking, using dyne:bolic and Netsukuku as historical and technical case studies. The project is informed by approximately 25 years of the student's work and experimentation with these technologies. Rather than treating cybersecurity solely as a defensive layer placed around conventional centralized infrastructure, the project asks whether security, privacy, resilience, and user autonomy can also be influenced by architectural choices that reduce dependence on centralized services.

dyne:bolic provides the endpoint and operating-system side of the study. Current project documentation describes it as a free GNU/Linux live operating system designed for multimedia production, portability, user freedom, privacy, and operation without mandatory cloud services or background communications. It also documents encryption support and an application-level network protection tool. Netsukuku provides the network-side case study. Its project documentation describes an experimental peer-to-peer mesh architecture intended to build and maintain routing in a decentralized manner, including the distributed ANDNA hostname system. These characteristics make the projects useful for examining a central cybersecurity question: what changes when control, trust, routing, and availability are distributed?

The project does not assume that decentralization automatically creates security. Instead, it evaluates both benefits and new risks. Decentralized designs may reduce single points of control or failure, but they can introduce difficult problems involving node trust, malicious participants, routing manipulation, key management, software supply chains, monitoring, recovery, and governance. The proposed laboratory therefore focuses on controlled, authorized experiments rather than attacks against public infrastructure.

The analysis is organized using the NIST Cybersecurity Framework (CSF) 2.0 functions of Govern, Identify, Protect, Detect, Respond, and Recover. The final product combines historical research, threat modeling, risk assessment, architecture diagrams, and a laboratory methodology. The goal is not to prove that one architecture is universally superior, but to demonstrate how cybersecurity professionals can evaluate decentralized systems using disciplined risk-management methods.

Executive Summary

Modern cybersecurity commonly assumes a centralized environment: centralized identity, centralized cloud services, centralized DNS, centralized logging, centralized administration, and centralized security operations. These arrangements can be efficient, but concentration also creates dependencies. A failure, compromise, policy decision, or outage affecting a central service can have consequences across many dependent users.

This project investigates an alternative perspective rooted in free software and decentralized networking. The student's approximately 25 years of work with dyne:bolic and Netsukuku provide a practical historical thread connecting operating-system freedom, portable computing, peer-to-peer networking, distributed routing, and user control.

The project has four objectives: first, document the historical and technical ideas represented by dyne:bolic and Netsukuku; second, translate those ideas into modern cybersecurity concepts; third, identify threats and risks introduced or reduced by decentralization; and fourth, demonstrate a controlled laboratory methodology for evaluating resilience and security properties.

The project uses NIST CSF 2.0 as its organizing framework. NIST CSF 2.0 is built around six concurrent Functions: Govern, Identify, Protect, Detect, Respond, and Recover. This provides a modern risk-management vocabulary for evaluating technologies that were developed under somewhat different historical assumptions.

The expected conclusion is deliberately balanced: decentralization is neither a universal security solution nor merely an obsolete idea. It is an architectural choice that changes the distribution of trust, failure, control, and attack surface. The cybersecurity value of such a system must therefore be measured against explicit threats, assets, assumptions, and operational requirements.

1. Research Questions, Thesis, and Scope

1.1 Primary Research Question

Can cybersecurity be strengthened by reducing dependence on centralized infrastructure and giving users greater control over their operating systems, data, and communications networks?

1.2 Secondary Research Questions

1. What cybersecurity properties are associated with a free, portable, user-controlled operating environment such as dyne:bolic?
2. What security and resilience properties are associated with decentralized peer-to-peer routing as represented by Netsukuku?
3. Which risks are reduced by decentralization, and which risks are merely moved to another layer?
4. How should trust, identity, routing integrity, availability, and recovery be evaluated in a distributed environment?
5. How can the NIST Cybersecurity Framework 2.0 be applied to a decentralized architecture?
6. What can modern cybersecurity practitioners learn from long-running free-software and decentralized-networking projects?

1.3 Thesis

Cybersecurity should be evaluated partly as an architectural property rather than exclusively as a collection of defensive products. dyne:bolic and Netsukuku demonstrate a long-running approach in which user control, free software, portable computing, decentralized networking, and distributed services are treated as design principles. These principles can reduce certain forms of centralized dependency and improve resilience in some scenarios, but they also introduce distinct risks involving trust, routing integrity, identity, monitoring, maintenance, and recovery. Therefore, decentralized cybersecurity should be evaluated through explicit threat models and measurable security outcomes rather than through the assumption that decentralization is inherently secure.

1.4 Scope

The project covers operating-system architecture, network architecture, trust, privacy, availability, resilience, identity, monitoring, incident response, and recovery. The laboratory scope is limited to systems owned by or explicitly authorized for the student and to isolated virtual or physical test environments.

1.5 Out of Scope

- Unauthorized scanning, exploitation, or traffic interception against third-party systems.
- Attempts to disrupt public networks or services.
- Claims that any historical project is production-ready for modern enterprise use without current validation.
- Claims about anonymity, censorship resistance, or security that cannot be supported by technical evidence.
- A full enterprise penetration test or compliance audit.

2. Historical Context: From Centralized Computing to User Sovereignty

The history of computing provides the first part of the project's context. The University of Florida history page supplied for this project describes the transition from human calculation to electronic computing and highlights ENIAC as a landmark in electronic digital computing. It also describes the importance of the transistor in the subsequent development of smaller and more practical computers.

That historical trajectory can be viewed as a gradual movement in the location of computing power. Early computing was concentrated in specialized facilities. Personal computing then moved computation into the hands of individuals. Networking connected those computers, while later cloud computing moved many functions back toward large centralized providers.

The important cybersecurity question is not whether centralization is good or bad. Centralization can improve manageability, patching, monitoring, and policy enforcement. The question is what dependencies are created when critical functions are concentrated in a small number of services or organizations.

2.1 Free Software and the Question of Control

Free software introduces a different model of control. Instead of treating software as an opaque product that the user may only execute, free-software principles emphasize the user's ability to study, modify, share, and improve software. From a security perspective, this creates potential advantages such as inspectability and reduced vendor lock-in, but it does not guarantee that software is secure.

A secure assessment must therefore separate the concepts of freedom, transparency, maintainability, and security. Open code can contain vulnerabilities. A project can have insufficient maintainers. Dependencies can introduce supply-chain risks. Conversely, an inspectable codebase can make independent review and community repair possible.

2.2 The Dyne.org Context

Current dyne.org documentation states that the dyne foundation's community originated online in 1999 and identifies diversity, open source, and ecology as guiding principles. The current dyne:bolic documentation describes the system as a free GNU/Linux distribution and emphasizes portability, privacy, user control, and the use of free software.

Sources: dyne:bolic Manual, dyne.org; dyne.org Foundation documentation.

3. dyne:bolic as an Endpoint Security Case Study

3.1 System Characteristics

The current dyne:bolic manual describes dyne:bolic IV as a 100% Free Software GNU/Linux operating system for multimedia production. It is distributed as a live system and can operate from removable media without requiring installation on the computer's internal disk. The documentation also describes the ability to boot from multiple media types and to store data in a portable file.

From a cybersecurity perspective, portability changes the assumptions surrounding endpoint ownership and persistence. A live environment can reduce dependence on a preexisting installed operating system, while portability can allow a user to move a known environment between compatible machines. These are architectural properties, not automatic security guarantees.

3.2 Privacy and Communications

The dyne:bolic manual states that the distribution does not include cloud services and does not establish background communications without the user's consent. It also documents support for strong encryption of private data through Tomb. These characteristics are relevant to data minimization, user control, and confidentiality.

3.3 Network Protection

The current getting-started documentation also describes OpenSnitch as a graphical firewall tool shipped with dyne:bolic and states that it blocks incoming and outgoing traffic by default until the user permits it. In the proposed project, this feature becomes an example of endpoint-level policy enforcement that can be tested and documented.

3.4 Security Advantages to Evaluate

- Reduced dependence on mandatory cloud services.
- Portable and potentially disposable endpoint environment.
- Ability to inspect and modify free software.
- Support for encrypted private data.
- User-visible network access decisions.
- Ability to operate on older hardware, potentially extending the useful life of existing systems.

3.5 Security Limitations to Evaluate

- A live operating system does not prevent a compromised application from being malicious.
- Physical access to removable media can create integrity and availability risks.
- Users may make insecure configuration decisions.
- Software provenance and update authenticity still matter.
- Hardware and firmware can remain outside the user's control.
- A portable system may complicate centralized enterprise management and monitoring.

Sources: dyne:bolic Manual and Getting Started documentation, dyne.org.

4. Netsukuku as a Network Security Case Study

4.1 Concept

Netsukuku is documented as an experimental peer-to-peer or mesh networking system intended to generate and sustain a decentralized network. Its project site describes a model in which nodes automatically participate in route construction and in which the network does not require a conventional centralized routing authority. The historical project documentation also describes ANDNA, a distributed hostname management system intended to replace hierarchical DNS within the Netsukuku environment.

4.2 Routing and Topology

Netsukuku documentation describes a hierarchical grouping of nodes into larger logical groups and a distributed approach to route management. The architecture documentation discusses neighbor detection, link-quality measurement, and QSPN, a routing mechanism used to explore the network and react to changes.

This is significant for cybersecurity because routing becomes a distributed trust and availability problem. A conventional network administrator may have a central routing configuration to inspect and control. A distributed network must instead reason about the correctness and trustworthiness of information exchanged among participants.

4.3 ANDNA

The ANDNA system is documented in both Netsukuku project materials and an academic paper as a distributed, non-hierarchical hostname-management system. This creates an instructive comparison with conventional DNS: the security question shifts from protecting a centralized naming hierarchy to protecting a distributed naming process.

4.4 Historical Relationship to dyne:bolic

The Netsukuku project site states that a public version was released in October 2006 and included ready to use inside the dyne:bolic GNU/Linux distribution. This relationship is central to this project because it connects the endpoint and network layers rather than treating them as unrelated technologies.

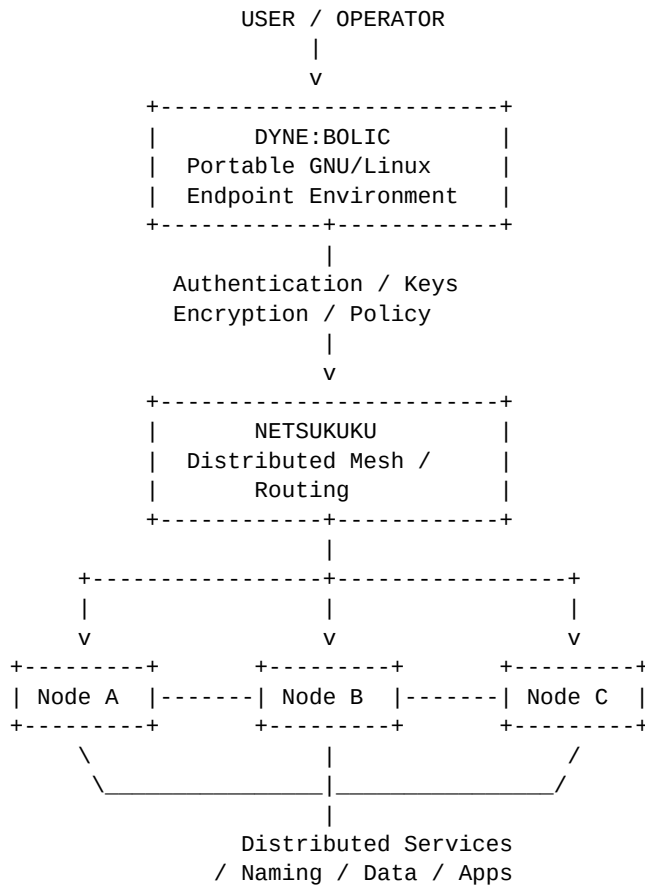
4.5 Important Caveat

Historical Netsukuku documentation explicitly warns that some versions were experimental, tested on relatively small scenarios, and not ready for mass deployment. That limitation is essential to preserve in an academic project. The purpose of this midterm is therefore to study the architecture and security questions, not to claim that historical Netsukuku is currently a production replacement for the public Internet.

Sources: Netsukuku project site; Dyne.org Netsukuku wiki; Lo Pumo, ANDNA paper.

5. Proposed Decentralized Cybersecurity Architecture

The proposed architecture combines the endpoint and network concepts into one analytical model. dyne:bolic represents the user-controlled endpoint layer. Netsukuku represents the decentralized routing layer. Cryptographic controls, identity mechanisms, monitoring, and recovery procedures are placed across both layers.



5.1 Security Design Principles

7. Minimize unnecessary centralized dependencies.
8. Treat every node and credential as potentially compromiseable.
9. Use cryptographic verification rather than assumed trust.
10. Separate availability from the security of any single node.
11. Make security decisions observable through logging and monitoring.
12. Design recovery procedures before an incident occurs.
13. Preserve user control while acknowledging operational responsibilities.

5.2 Centralized vs. Decentralized Model

Area	Centralized Model	Decentralized Model
Control	Central authority	Distributed participants
Routing	Centralized or hierarchical	Distributed routing

	infrastructure	
Failure	Central dependency may create broad impact	Failures may be localized if redundancy exists
Trust	Provider/institutional trust	Distributed and cryptographic trust
Monitoring	Centralized collection is easier	Monitoring is more distributed
Governance	Central policy	Distributed governance
Recovery	Central restoration mechanisms	Node-level and distributed recovery
Attack Surface	Concentrated services can be high-value targets	More nodes and protocols can create broader attack surface

6. Asset Inventory and Threat Model

6.1 Assets

Asset	Security Objective	Potential Impact
Endpoint OS	Integrity, confidentiality, availability	Compromise of user environment
Private files	Confidentiality, integrity	Data exposure or destruction
Cryptographic keys	Confidentiality, integrity	Impersonation or data loss
Network routes	Integrity, availability	Traffic disruption or manipulation
Node identity	Authenticity	Unauthorized participation
Distributed names	Integrity, authenticity	Misrouting or impersonation
Logs	Integrity, availability	Loss of investigative evidence
Backup/recovery data	Availability, integrity	Extended outage or data loss

6.2 Threat Actors

- Compromised legitimate node.
- Malicious node intentionally providing incorrect routing or naming information.
- External attacker attempting to compromise endpoints.
- Credential or key thief.
- Software supply-chain attacker.
- Availability attacker attempting to isolate or overwhelm nodes.
- Physical attacker with access to equipment or removable media.

6.3 Threat Modeling Questions

14. What must an attacker compromise to influence routing?
15. How can a node prove that another node is legitimate?
16. What happens when a trusted node becomes malicious?
17. Can an attacker create multiple identities?
18. How is a compromised key revoked?
19. How can operators detect abnormal routing behavior?
20. How does the system recover after node compromise?

7. Risk Assessment and Risk Matrix

The risk assessment uses a simple likelihood-impact model. The ratings are preliminary and should be revised after laboratory evidence is collected. The purpose is to prioritize testing rather than to pretend that a numerical score is a precise measurement.

Risk	Likelihood	Impact	Priority	Proposed Control
Compromised endpoint	High	High	Critical	Least privilege, encryption, endpoint monitoring, recovery image
Malicious/compromised node	Medium	High	High	Node authentication, route validation, isolation
Routing manipulation	Medium	High	High	Integrity checks, anomaly detection, redundant paths
Key compromise	Medium	Critical	Critical	Protected key storage, rotation, revocation
Supply-chain compromise	Medium	High	High	Source verification, signed releases, hash verification
Loss of a node	High	Medium	High	Redundant routing and documented recovery
Distributed naming abuse	Medium	High	High	Validation, registration controls, monitoring
Insufficient monitoring	High	Medium	High	Centralized or federated logging
Physical theft	Medium	Medium	Medium	Encryption and physical controls

Configuration error	High	Medium	High	Baseline configurations and change control
---------------------	------	--------	------	--

7.1 Risk Treatment Strategy

- Critical risks receive immediate mitigation or explicit acceptance by the project owner.
- High risks become primary laboratory test cases.
- Medium risks are monitored and addressed through procedural or technical controls.
- Low risks are documented and reviewed periodically.

7.2 Key Observation

The matrix demonstrates the central theme of the project: decentralization may reduce some forms of centralized dependency while increasing the importance of distributed trust, node integrity, cryptographic identity, and monitoring. The security problem has not disappeared; its location has changed.

8. NIST Cybersecurity Framework 2.0 Mapping

NIST CSF 2.0 organizes cybersecurity outcomes into six Functions: Govern, Identify, Protect, Detect, Respond, and Recover. Unlike a rigid checklist, the framework provides a common language for understanding and communicating cybersecurity risk.

NIST Function	Application to Proposed Architecture
Govern	Define ownership, acceptable risk, security policies, responsibilities, and laboratory rules.
Identify	Inventory nodes, software, keys, data, dependencies, routes, and critical services.
Protect	Use authentication, encryption, least privilege, secure configuration, and controlled updates.
Detect	Monitor node behavior, routing changes, authentication events, and endpoint alerts.
Respond	Isolate compromised nodes, revoke credentials, investigate evidence, and communicate impact.
Recover	Restore trusted software, replace compromised keys, rejoin nodes, validate integrity, and document lessons learned.

8.1 Why CSF 2.0 Matters Here

The framework provides a modern cybersecurity vocabulary without requiring the project to abandon its historical subject matter. A decentralized system can still be governed, its assets can still be identified, protections can still be implemented, attacks can still be detected, incidents can still be handled, and systems can still be recovered.

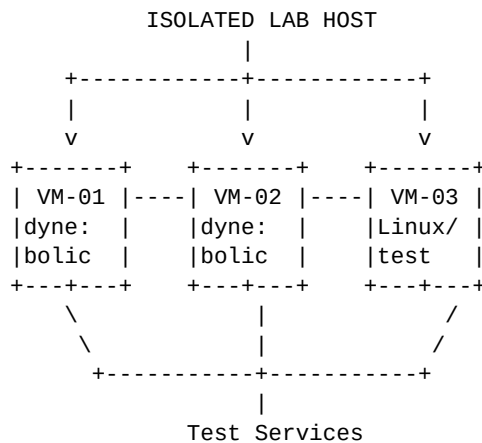
Source: NIST Cybersecurity Framework 2.0, NIST CSWP 29, February 2024.

9. Laboratory Methodology

9.1 Purpose

The laboratory is designed to evaluate security and resilience claims in a controlled environment. It is not intended to attack public infrastructure. All systems will be owned by the student, provided by the boot camp, or explicitly authorized for testing.

9.2 Proposed Environment



9.3 Lab Variables

Variable	Measurement
Node availability	Reachability before/after a node failure
Route convergence	Time and route changes after topology change
Authentication	Successful/failed identity validation
Integrity	Detection of altered configuration or data
Monitoring	Time to detect abnormal behavior
Recovery	Time to restore a trusted node
Resource use	CPU, memory, and network overhead

9.4 Experiment 1 — Baseline

21. Document the test topology.
22. Record software versions and configurations.
23. Establish normal node-to-node communication.
24. Capture baseline CPU, memory, and network measurements.
25. Record expected routing and naming behavior.

9.5 Experiment 2 — Node Failure

26. Remove one authorized test node from the network.

27. Measure which paths remain available.
28. Observe route changes and convergence behavior.
29. Record service availability.
30. Restore the node and document recovery.

9.6 Experiment 3 — Compromised-Node Simulation

In a closed laboratory, configure one test node to produce deliberately incorrect or abnormal behavior that represents a compromised participant. The purpose is to test detection and containment, not to develop or deploy malware.

31. Establish the normal behavior baseline.
32. Introduce the controlled abnormal condition.
33. Capture logs and network observations.
34. Determine whether other nodes detect the condition.
35. Isolate the test node.
36. Restore a known-good configuration.
37. Document lessons learned.

9.7 Experiment 4 — Recovery

38. Destroy or invalidate a disposable test-node configuration.
39. Restore from a known-good image or configuration.
40. Reestablish keys or credentials as appropriate.
41. Verify software and configuration integrity.
42. Rejoin the node to the test environment.
43. Measure recovery time.

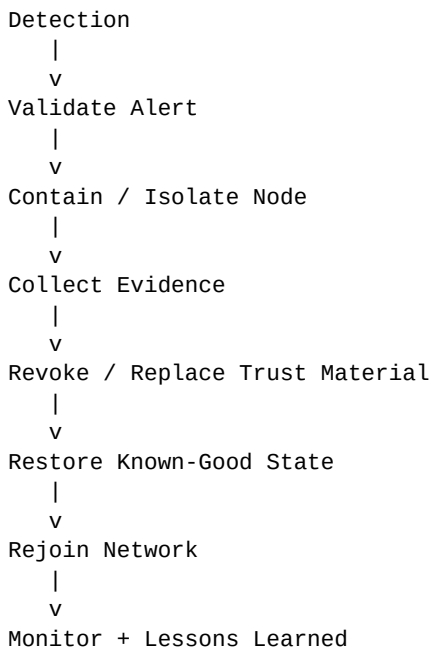
10. Monitoring, Detection, and Incident Response

10.1 Monitoring Strategy

- Authentication events.
- Node joins and departures.
- Unexpected route changes.
- Changes to security configuration.
- Unexpected network connections.
- Integrity changes to critical files.
- Resource-consumption anomalies.
- Repeated failed authentication or authorization events.

10.2 Incident Scenario

Scenario: a legitimate node is suspected of compromise after producing abnormal network behavior.



10.3 Incident Response Questions

44. How quickly can the abnormal node be identified?
45. Can it be isolated without disabling unrelated nodes?
46. What evidence remains available after isolation?
47. How are credentials or keys revoked?
48. How is a restored node proven trustworthy?
49. How is the incident documented?

10.4 Evidence Collection

Evidence should be collected in a manner appropriate to the course's laboratory rules. Useful artifacts include system logs, timestamps, configuration snapshots, routing observations, authentication records, hashes of relevant files, and screenshots. The project should distinguish between observations, interpretations, and conclusions.

11. Security Controls and Defensive Design

11.1 Endpoint Controls

- Use known-good images and verify their integrity before deployment.
- Apply least privilege.
- Encrypt sensitive persistent data.
- Restrict unnecessary network access.
- Monitor security-relevant events.
- Maintain a tested recovery process.

11.2 Network Controls

- Authenticate participating nodes where supported by the implementation.
- Validate routing information and monitor unexpected changes.
- Maintain redundant paths where possible.
- Separate management functions from ordinary traffic.
- Record topology changes for investigation.

11.3 Cryptographic Controls

Cryptography should be treated as a trust mechanism rather than as a decorative feature. The project should document what is encrypted, what is authenticated, where keys are stored, how keys are rotated, and what happens after compromise.

11.4 Supply-Chain Security

Free software does not remove supply-chain risk. The project should consider source provenance, release signatures where available, checksums, package dependencies, build processes, and the trust relationship between the user and software distribution channels.

11.5 Recovery Controls

- Known-good images.
- Configuration backups.
- Documented key-replacement procedures.
- Offline or otherwise protected recovery material.
- Regular restoration testing.

12. Practitioner Perspective: Approximately 25 Years of Experimentation

A central purpose of this project is to connect formal cybersecurity education with the student's long-term practical relationship with free software and decentralized networking. The student's work with dyne:bolic and Netsukuku spans approximately 25 years. This experience provides a longitudinal perspective that is difficult to obtain from a short-term laboratory exercise.

12.1 From Technology to Security Thinking

Working with free software changes the relationship between the user and the computer. The computer becomes something that can be inspected, modified, repaired, and repurposed rather than only consumed as a packaged product. Working with decentralized networking extends that question from the endpoint to the network: who controls routing, who provides names, and what happens when infrastructure is unavailable?

12.2 Lessons Across Time

One lesson is that architectural independence has costs. A user-controlled system requires the user or community to understand more of the underlying technology. A decentralized network requires participants to understand trust and failure without relying on one administrator to solve every problem.

Another lesson is that resilience should not be confused with invulnerability. A distributed system can continue operating through some failures while remaining vulnerable to malicious participants, software flaws, physical destruction, or cryptographic compromise.

12.3 Why This Matters in a Cybersecurity Boot Camp

Cybersecurity professionals are frequently trained to select and operate tools. This project adds another layer of thinking: evaluate the assumptions built into the architecture itself. The goal is not to reject enterprise security tools, but to understand where architecture can either create or reduce security dependencies.

12.4 Personal Reflection Prompts

- What did I originally hope decentralized computing would accomplish?
- Which assumptions from 25 years ago still hold?
- Which assumptions have been disproved by experience?
- What security problems did I underestimate?
- What would I redesign today?
- Which lessons from this work apply to modern enterprise cybersecurity?

13. Analysis, Limitations, and Ethical Considerations

13.1 Decentralization Is Not Automatically Secure

The strongest conclusion available from this project is not that decentralized systems are inherently safer. Instead, decentralization changes where security work must occur. Centralized environments concentrate administrative authority and monitoring. Distributed environments spread responsibility across nodes, identities, protocols, and participants.

13.2 Operational Complexity

Distributed systems can be difficult to troubleshoot because there may be no single authoritative state. A routing problem can result from several interacting nodes. A trust problem can involve multiple identities. Recovery may require coordinated changes across participants.

13.3 Monitoring Challenge

Centralized security operations can collect logs into a single analysis environment. A decentralized system may require federated or distributed monitoring. This can create visibility gaps unless the architecture deliberately provides telemetry and evidence collection.

13.4 Governance

A decentralized technical architecture does not eliminate governance. It changes the question from 'Who is the administrator?' to 'What rules govern participation, trust, software updates, naming, and conflict resolution?' This is a cybersecurity issue because poorly defined governance can become an attack surface.

13.5 Ethical Boundaries

All experiments in this project should be conducted only against authorized systems. The student should use isolated virtual machines or hardware owned by the student or provided for the exercise. No public routing infrastructure, third-party device, ISP network, or external service should be disrupted or tested without explicit authorization.

14. Implementation and Evaluation Roadmap

Phase	Activities	Evidence
Phase 1 — Prepare	Define scope, build lab, inventory assets	Topology diagram, asset list
Phase 2 — Baseline	Verify normal operation and measurements	Baseline logs and screenshots
Phase 3 — Failure Testing	Remove test nodes and observe behavior	Timing and connectivity results
Phase 4 — Security Testing	Simulate authorized abnormal behavior	Detection/containment evidence
Phase 5 — Recovery	Restore trusted configurations	Recovery measurements
Phase 6 — Analysis	Compare results with threat model	Risk updates
Phase 7 — Reporting	Write findings and recommendations	Final report and presentation

14.1 Success Criteria

- The laboratory topology is reproducible.
- Baseline behavior is documented.
- At least one controlled node-failure experiment is completed.
- At least one controlled security-anomaly experiment is completed.
- Detection and containment steps are documented.
- Recovery is demonstrated and measured.
- Risks are updated based on evidence rather than assumptions.
- The final architecture is mapped to NIST CSF 2.0.

14.2 Expected Findings

The project expects to find that decentralized architecture can provide resilience and independence from selected centralized dependencies, but that these benefits depend on implementation quality, node diversity, cryptographic trust, monitoring, and recovery. The laboratory may also demonstrate that a theoretically resilient network can still be operationally fragile if participants cannot detect or recover from compromise.

15. Conclusion

This project began with a historical question about computing and developed into a cybersecurity question about architecture. The evolution from centralized machines to personal computing, networked systems, cloud services, and decentralized alternatives demonstrates that the location of control is itself a technical design decision.

dyne:bolic represents one side of this exploration: a free, portable GNU/Linux environment designed around user control, privacy, and independent media production. Netsukuku represents the network side: an experimental effort to construct decentralized routing and distributed naming without depending on conventional centralized infrastructure. Their historical relationship makes them particularly valuable as a combined case study.

The cybersecurity lesson is not that decentralization replaces conventional security. Instead, decentralized architecture changes the security model. It can reduce some single points of dependency while introducing new problems around trust, identity, routing, monitoring, maintenance, and recovery.

The student's approximately 25 years of experience provides the project with a longitudinal perspective. Concepts that may appear new in contemporary discussions—user sovereignty, resilient networking, distributed services, privacy, and reducing dependency on centralized platforms—have been explored by free-software and decentralized-networking communities for decades.

The final conclusion is therefore deliberately practical: cybersecurity professionals should learn to evaluate both defensive controls and architectural assumptions. A system should be judged by its actual threat model, evidence, operational requirements, and recovery capabilities. Decentralization may be one tool in that larger cybersecurity design space, particularly when resilience, user control, and independence from centralized services are important requirements.

16. Midterm Presentation Outline

Recommended length: 12–15 minutes.

50. Title and central question — 45 seconds.
51. Why this project is different — 1 minute.
52. 25-year background — 1 minute.
53. Historical evolution of computing — 1 minute.
54. dyne:bolic case study — 1.5 minutes.
55. Netsukuku case study — 1.5 minutes.
56. Centralized vs. decentralized security — 1 minute.
57. Architecture diagram — 1 minute.
58. Threat model and risk matrix — 1 minute.
59. Laboratory methodology — 1.5 minutes.
60. NIST CSF 2.0 mapping — 1 minute.
61. Expected/observed findings — 1 minute.
62. Limitations and lessons learned — 45 seconds.
63. Conclusion — 45 seconds.

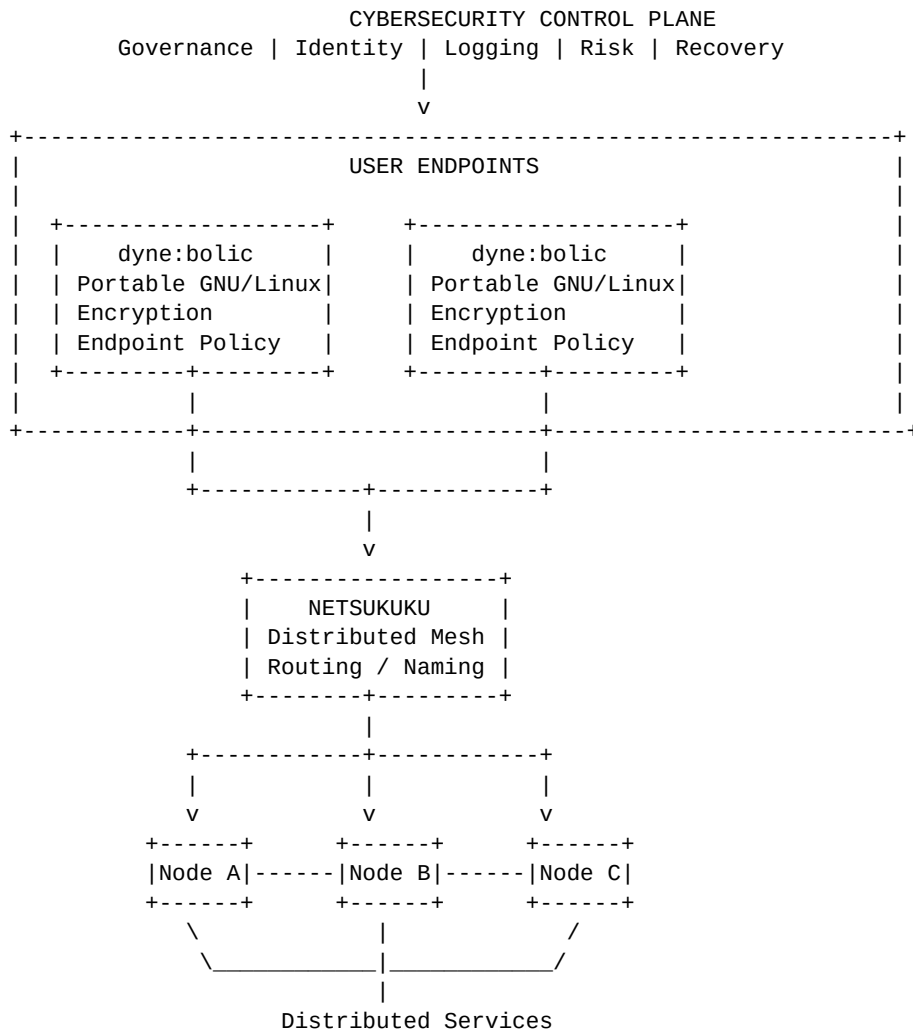
16.1 Suggested Presentation Opening

“For the last approximately 25 years, I have been working with ideas that are now being discussed again under modern cybersecurity terms: user sovereignty, free software, privacy, resilient networking, and decentralization. This project asks what happens when we stop treating those ideas as philosophy and evaluate them as cybersecurity architecture.”

16.2 Suggested Closing

“The point of this project is not to claim that decentralized systems are automatically secure. The point is that architecture determines where trust, control, failure, and responsibility live. Cybersecurity professionals should be able to evaluate those choices instead of assuming that centralization is the only model.”

Appendix A — Architecture Diagram



Appendix B — Risk Matrix

Impact \ Likelihood	Low	Medium	High
Low	Low	Low	Medium
Medium	Low	Medium	High
High	Medium	High	Critical
Critical	High	Critical	Critical

Appendix C — Laboratory Evidence Checklist

- Lab topology screenshot.
- Operating-system/version information.
- Network configuration record.

- Baseline connectivity evidence.
- Baseline resource measurements.
- Node-failure results.
- Abnormal-behavior detection results.
- Isolation/containment evidence.
- Recovery evidence.
- Updated risk matrix.
- Final lessons learned.

Appendix D — References

64. University of Florida, Institute for Learning in Retirement/Journalism resources, “History of Computers.” <https://iml.jou.ufl.edu/CARLSON/History/computers.htm>
65. Dyne.org, “Dynebolic Manual.” <https://dyne.org/docs/dynebolic/>
66. Dyne.org, “Getting Started with dynebolic.” https://dyne.org/docs/dynebolic/getting_started/
67. Dyne.org, “Dynebolic.” <https://dyne.org/dynebolic/>
68. Dyne.org, “Netsukuku.” <https://lab.dyne.org/Netsukuku>
69. Netsukuku project, “Netsukuku — P2P mesh routing.” <https://netsukuku.freaknet.org/>
70. Netsukuku project, “Netsukuku source documentation.” https://netsukuku.freaknet.org/sourcedocs/main_doc/index.html
71. Lo Pumo, Andrea. “ANDNA: the distributed hostname management system of Netsukuku.” arXiv:0705.0820. <https://arxiv.org/abs/0705.0820>
72. Lo Pumo, Andrea. “Overview of the Netsukuku network.” arXiv:0705.0815. <https://arxiv.org/abs/0705.0815>
73. NIST, “The NIST Cybersecurity Framework (CSF) 2.0.” NIST CSWP 29, February 26, 2024. <https://www.nist.gov/publications/nist-cybersecurity-framework-csf-20>